



统一微架构中间表示 VulSim & 冻源模拟器 相关工作汇总整理

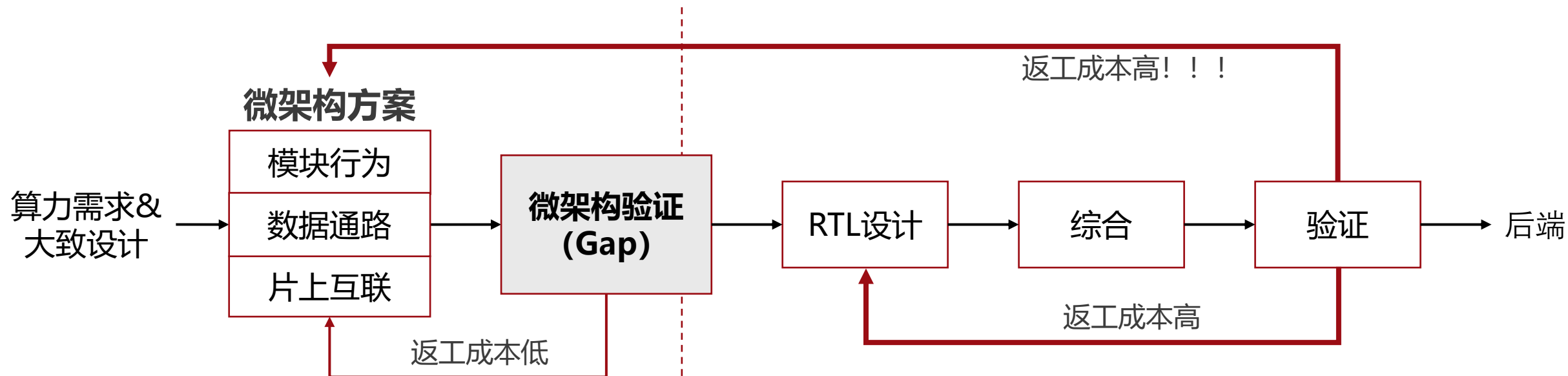
孟成真

山东大学

2026/06/12

1. 研究背景与问题

1. 新挑战：早期微架构决策需要可信验证手段



使用性能模型、GEM5、SystemC 等工具

迭代灵活，但语义细节不完整

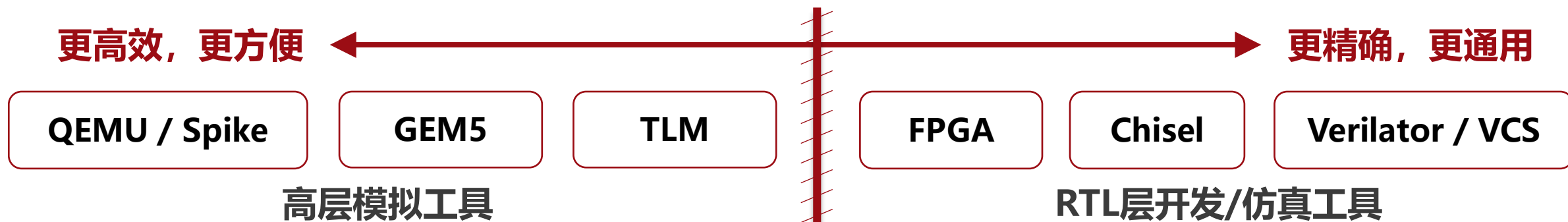
使用Verilog、System Verilog、Chisel等工具

语义准确完整，但迭代成本高

需要更完善的 Pre-RTL 阶段微架构建模与验证能力：

1. 能反映周期级微架构行为
2. 能在设计完成前发现性能和结构问题
3. 能让模型自然走向 RTL 实现，而不是推倒重写

2. 挑战的根本原因：跨抽象层级带来的工具断层



高层模型 与 RTL实现 是两套割裂的世界

模型割裂
高层模型和RTL是两份
结构完全不同的代码

语义割裂
高层软件执行语义和硬件
周期语义不统一

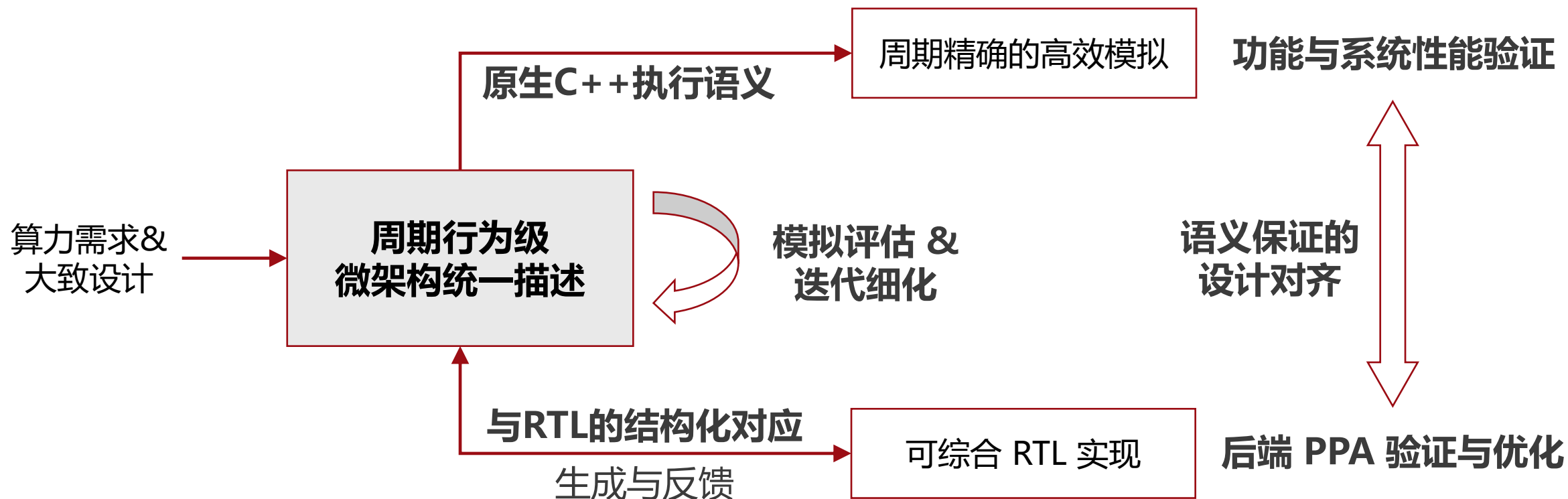
团队割裂
软件架构团队和硬件RTL
团队协作成本高

解决方法:

让同一份微架构描述贯穿整个设计流程

打通高层模型与 RTL 实现之间的抽象层级壁垒

3. 核心思想：让同一份微架构描述贯穿整个设计流程



贯穿全流程的周期行为级微架构统一描述模型，兼顾：

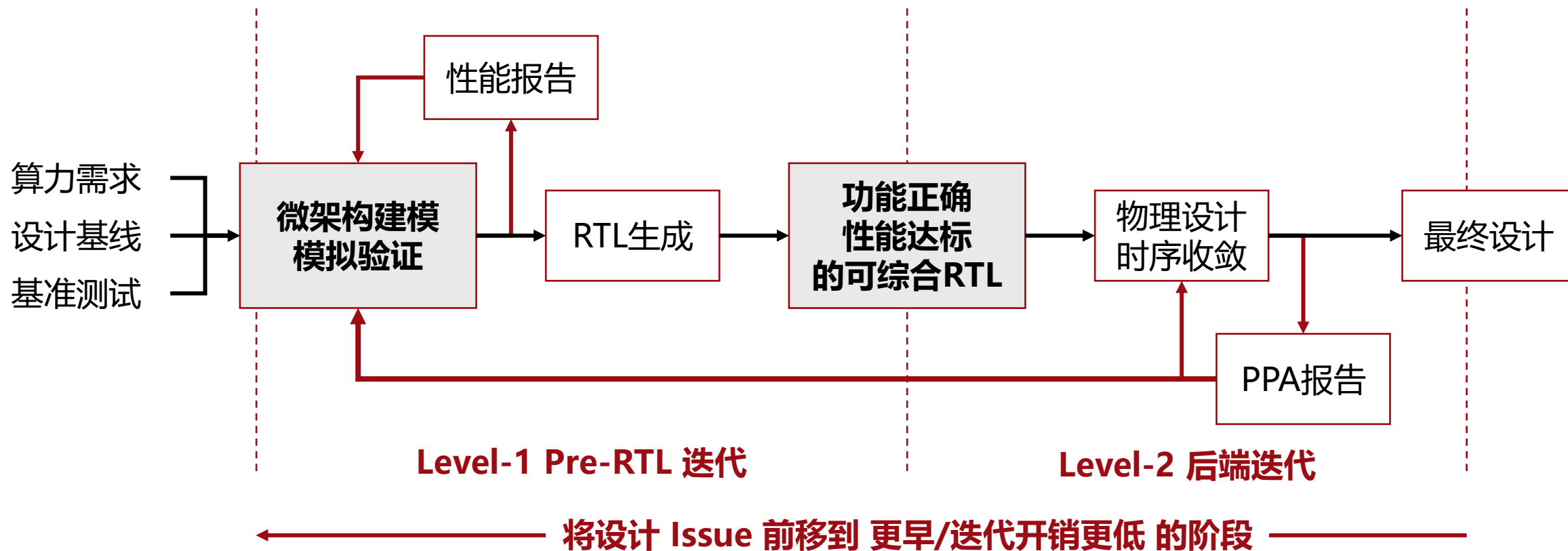
可执行性
像软件一样快速开发、
高效运行

周期保真
保留准确的模块层次、
周期边界和状态更新

可实现性
像硬件设计一样能够映射到
RTL和后端流程

项目地址：<https://gitee.com/dongshan-community/luo-yuan-sim>

4. 新开发范式：Pre-RTL 微架构设计、验证与迭代



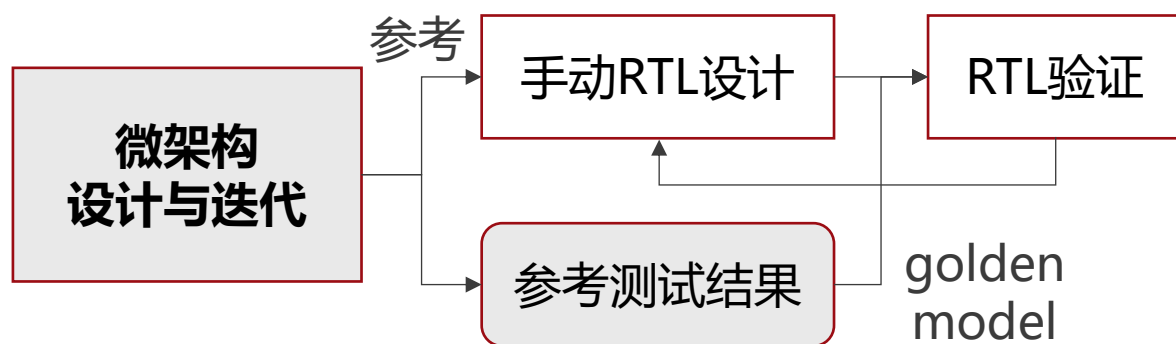
相比传统流程的核心优势：

1. 微架构问题更早暴露
2. 软件/架构/硬件团队围绕同一模型协同
3. 多核系统仿真速度比并行 Verilator 快 >40倍
4. 关键信号与 RTL 波形级一致性 >99%

5. 工程落地：实际应用模式与部署方案

部分应用

高保真度的微架构建模与迭代

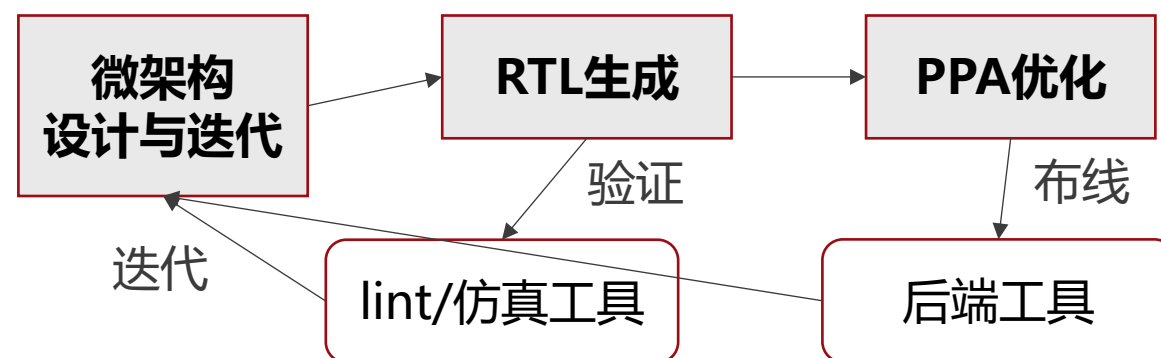


特点：使用不可综合C++描述行为
快速搭建，高效运行，敏捷验证

目的：快速建立周期精确的微架构模型
验证、评估微架构设计与参数选择
为 RTL 实现提供可执行 spec 和 golden model

全量应用

行为模型到 RTL 的闭环设计



特点：使用可综合C++语法子集描述行为
从 RTL-first 变为 μ arch-first 高效闭环流程

目的：从行为级模型自动生成可综合 RTL
将功能验证、RTL 生成、PPA 分析纳入统一流程
基于微架构修改的自动化重新生成与回归验证

6. 本质区别：一个新的硬件设计抽象层级

VS. 架构/系统模拟器
(如 GEM5)

快速估计/评估系统行为

VS.

让微架构本身成为可测试、可演进的设计对象

VS. RTL 仿真器
(如 Verilator)

验证已有 RTL 是否正确

VS.

避免过早进入 RTL 阶段带来的验证成本

VS. 高层次 RTL 生成器
(如 VitisHLS)

从高层代码黑盒生成 RTL

VS.

保持高层行为模型与RTL实现结构对应

VS. HDL 生成器
(如 Chisel)

软件语言描述硬件组件如何构造

VS.

软件语言定义硬件的行为和交互流程

准确定义：一种描述周期级数字硬件行为的设计中间层

用统一模型贯通微架构建模、快速执行验证与可综合 RTL 生成

2. VUL 统一微架构中间表示概述

1. 项目概述

Vul (Visualized-Unified Layer) 可视化统一中间层

Vul前端

VulCPP Project

Vul后端

SimGen

RTLGen

目标代码

可构建C++代码

可综合
SystemVerilog

周期精确的高效
仿真，用于功能
和性能验证

可综合的RTL代
码，用于敏捷硬
件实现

Vul基础组件库

并行模拟组件库

Vul波形与调试库

FASE SE组件库

组件库

5-Stage Pipeline
CPU

Out-of-Order
CPU

SIMT GPGPU

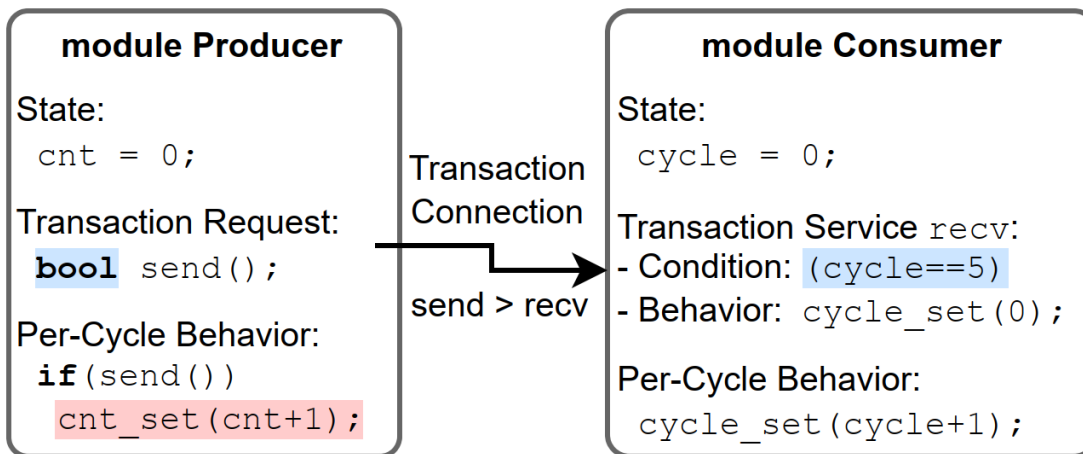
RISC-V TPU

RISC-V
Many-Cores

Heterogeneous
System

示例项目 (Working in progress)

2. 核心思想：写行为，而非结构



行为级硬件模型：

硬件模块 = 模块层次结构

+ 模块内部状态

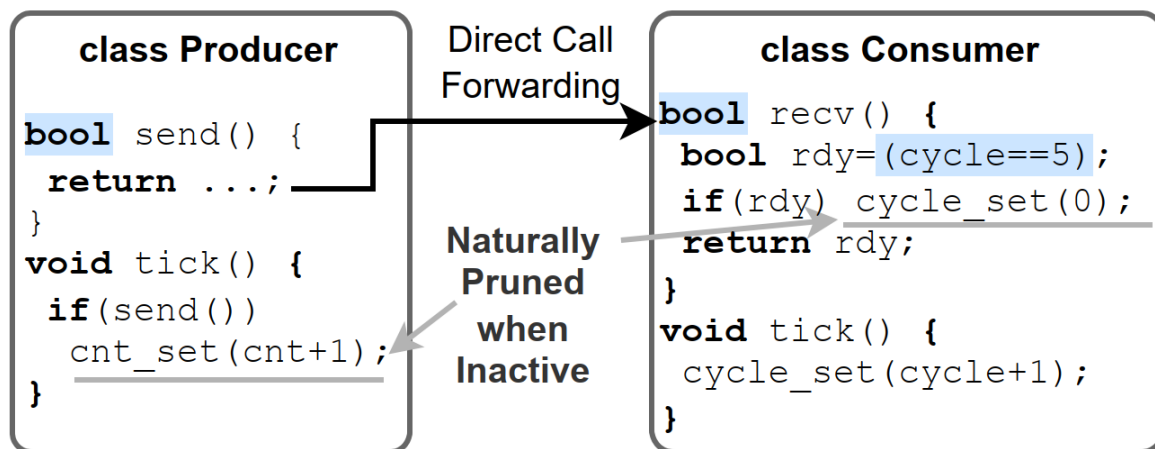
+ 行为过程描述

行为分为：

1. 每周期固定进行的行为

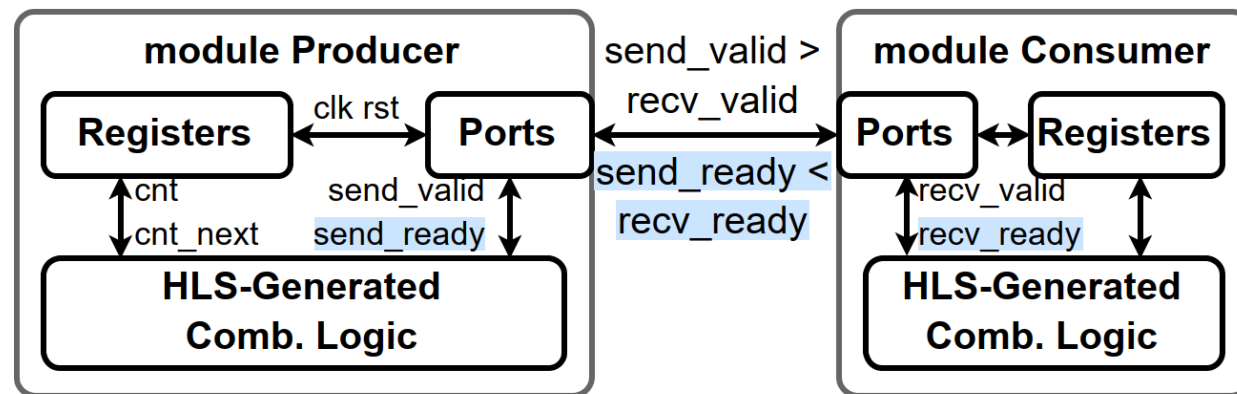
2. 通过事务调用触发的行为

↓ SimGen 代码重组



可构建的 C++

↓ RTLGen 代码翻译



可综合的 RTL

3. 相关工作：SystemC与事务层模型TLM

SystemC是基于C++语言的硬件建模库。TLM (Transaction Layer Model) 是其中一个抽象层级

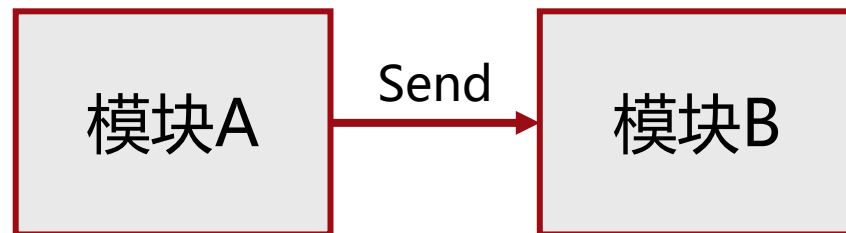
抽象层级	精度	目标场景
功能级	无时序信息	算法/功能模型验证
事务级	仅事务交互有延迟	粗略的微架构建模或通信性能验证
周期级	接近RTL的周期精度	微架构验证

使用SystemC时，需要先选定一个目标抽象层级然后进行建模。仅符合规范的周期级建模可以被HLS

TLM中将 信号束与交互 抽象为一个个事务，描述为带延迟的函数调用：

C++ Code:

```
Payload d;  
d.addr = 0x1000;  
d.data = 0xABCD;  
B.send(d, delay);
```



阻塞式事务：

- 立即完成+仿真延迟指定时间

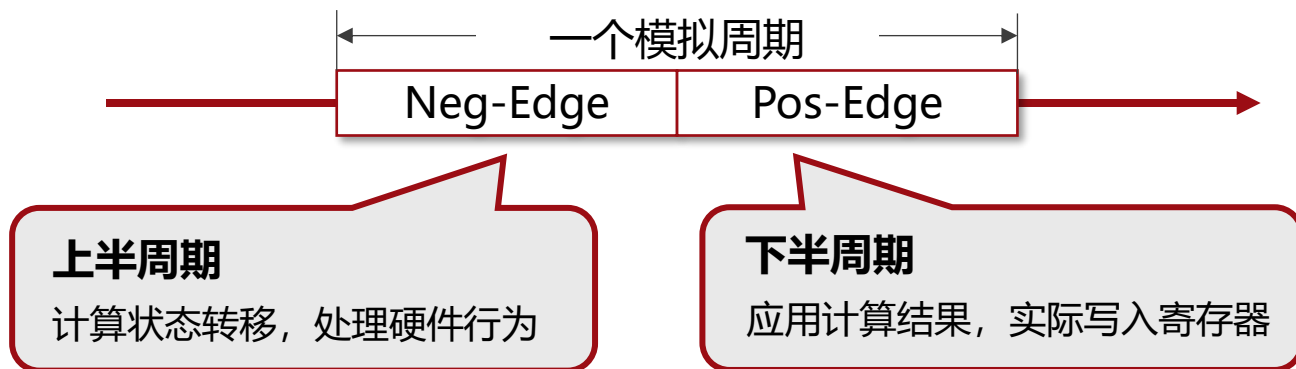
非阻塞式事务：

- 前向 (forward) 反向 (backward)
- 分阶段的行为建模与延迟

4. 相关工作：与 TLM 的差异：混合事务+周期建模

混合事务+周期建模方法：兼顾事务级模型的高仿真效率和表达能力+周期级模型的周期精确

- **周期驱动的仿真模式：**按照仿真周期同步更新所有硬件的本周期行为



优点1：直接对应RTL模型中的组合逻辑与寄存器

优点2：不同组件之间不存在数据冲突，提升并行性

- **事务描述的模块间交互：**仅保留单周期事务作为模块间信号交流的抽象

```
void A::neg_edge() {  
    if (b->send(xxx)) {  
        // handshake success  
    } else {  
        // b not ready  
    }  
}
```



```
bool B::send(uint64 d) {  
    if (ready condition) {  
        // handshake success  
        return true;  
    } else {  
        return false;  
    }  
}
```

5. 前端编程模型示例

Producer

```
REGISTER(cycle, uint8_t) {
    cycle = 0;
}
REGISTER(count, uint8_t) {
    count = 0;
}

REQUEST_READY(send, ARG(uint8_t) d);

TICK_IMPL() {
    cycle.setnext(cycle + 1);
    if (send(cycle)) {
        count.setnext(count + 1);
    }
}
```

Prod: 每周期尝试调用send,
成功时记录count
Cons: 偶数周期接受recv,
转发到output

Consumer

```
REGISTER(cycle, uint8_t) {
    cycle = 0;
}
REGISTER(sum, uint8_t) {
    sum = 0;
}

REQUEST(output, ARG(uint8_t) s);

SERVICE_READY(
    recv,
    (cycle & 1) == 0,
    ARG(uint8_t) d
) {
    sum.setnext(sum + d);
    output(sum);
}

TICK_IMPL() {
    cycle.setnext(cycle + 1);
}
```

TopModule

```
REQUEST(output, ARG(uint8_t) s);

CHILD_INSTANCE(Producer, prod);
CHILD_INSTANCE(Consumer, cons);

CONNECT_CR_CS(prod, send, cons, recv);

CONNECT_CR_R(cons, output, output);
```

TopModule:
实例化 Prod 和 Cons
连接 prod.send -> cons.recv
连接 cons.output -> this.output

6. 前端编程模型示例

Test Block

```
GLOBAL() {
    uint64_t tick_count = 0;
}

SERVICE(output, ARG(uint8_t) s) {
    printf("%ld: Output: %u\n", tick_count, s);
}

SIMULATION() {
    for (
        tick_count = 0;
        tick_count < 20;
        ++tick_count
    ) {
        sim_execute();
        sim_commit();
    }
}
```

全局变量保存当前模拟周期数

Service output 对应 TopModule.output

运行 20 周期的模拟

TopModule

```
REQUEST(output, ARG(uint8_t) s);

CHILD_INSTANCE(Producer, prod);
CHILD_INSTANCE(Consumer, cons);

CONNECT_CR_CS(prod, send, cons, recv);

CONNECT_CR_R(cons, output, output);
```

仿真运行结果

```
(base) null@cislc-nbplus:~/vulsim/vulsim/build$ ./simout/TopModule
0: Output: 0
2: Output: 0
4: Output: 2
6: Output: 6
8: Output: 12
10: Output: 20
12: Output: 30
14: Output: 42
16: Output: 56
18: Output: 72
```


3. VUL 模型细节与实现

1. Vul 模型 – C++ 代码 – RTL 对应关系

Vul模型元素	描述	C++元素	SystemVerilog元素
CONSTANT	整数常量	constexpr int64 xxx = xxx;	localparam longint
STRUCT	相关数据组成的结构体	struct	平铺所有信号
MODULE	模块定义	template<int64 WID=4> class ModName	module ModName #(parameter WID = 4);
PARAMETER	模块参数常量		
INSTANCE	子模块实例	unique_ptr<ChildMod<N>>	ChildMod #(N)
REGISTER	跨周期保留的内部状态	VulReg<Type> name 成员变量	reg [N:0] name
WIRE	每周期重置的内部状态	Type name 成员变量	wire [N:0] name
REQUEST	模块的事务请求端口	private 成员函数	input xxx_valid, input [xx:0] xxx_data, output xxx_ready
SERVICE	模块的事务服务端口	public 成员函数	(REQUEST 反向)
CONNECT	事务端口间的连接	静态绑定的函数调用	wire 端口连线

2. Vul 事务模型: Request & Service

Request & Service 以事务形式描述了模块间基于组合逻辑的信号交互。

RRequest & RService 是带有Ready信号的衍生组件，其区别在于事务返回值为bool而不是void。

Vul 模型中一个事务可以带有若干 参数（调用方驱动接受方）和 回复（接受方驱动调用方）：

例1: **void** request_**regread**(**uint16** idx, **uint64** * **value**) ;

无Ready

事务名

参数

回复

对应的RTL生成代码框架

```
output regread_valid,  
output [15:0] regread_idx,  
input [63:0] regread_value,
```

例2: **void** request_**redirect**(**uint64** newpc) ;

无Ready

事务名

参数

```
output redirect_valid,  
output [63:0] redirect_newpc,
```

例3: **bool** request_**axi_ar_send**(**AXIARBundle** & **ar**) ;

有Ready

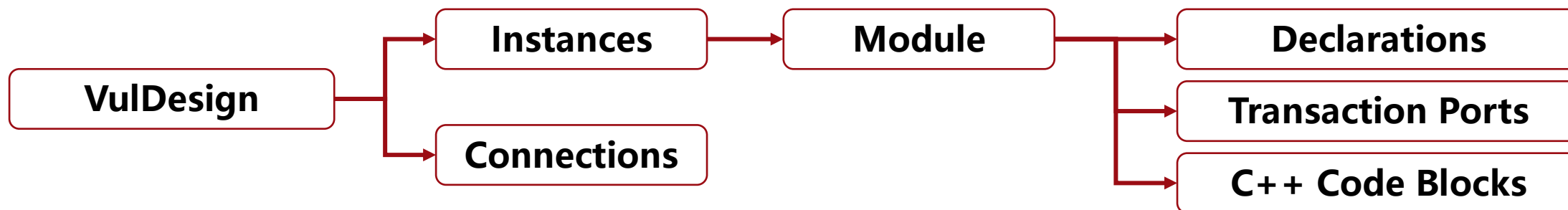
事务名

参数

```
output axi_ar_send_valid,  
output [31:0] axi_ar_send_ar_addr,  
output [7:0] axi_ar_send_ar_len,  
... ..  
input axi_ar_send_ready,
```

无Ready无回复的Request（例2）可以连接任意个匹配的Service或悬空；其他Request连且仅能连接一个Service

3. Vul 事务模型: Instance & Connection



一份 Vul 设计由一组实例 (Instance) 和它们之间的连接 (Connection) 组成。

一个 Instance 是一个 Module 的实例化, 也是 Vul 处理硬件执行语义的单位。

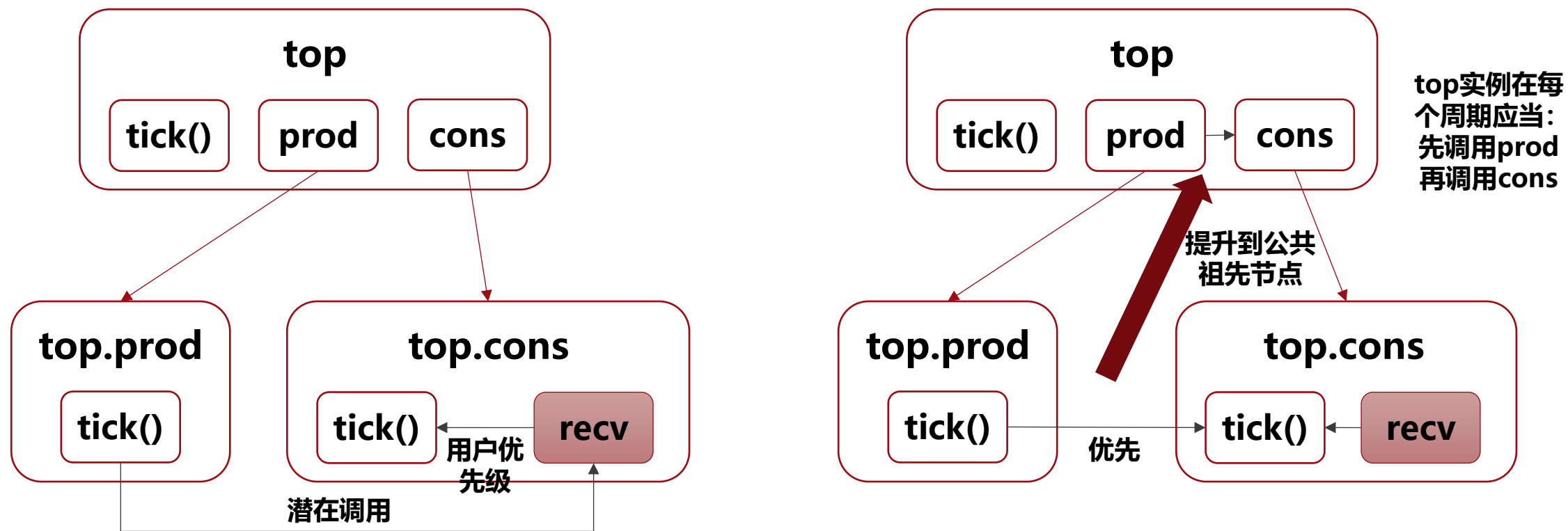
Vul 模型中共有四类 Connection:

- **CR-CS:** 从一个子实例的请求端口 到 另一个子实例的服务端口。
- **CR-R:** 从一个子实例的请求端口 到 父模块的请求端口, 代表事务向上级 bypass。
- **CR-S:** 从一个子实例的请求端口 到 父模块的服务端口, 代表事务被父模块内部响应。
- **S-CS:** 从父模块的服务端口 到 子实例的服务端口, 代表事务向下级 bypass。

4. Vul 执行调度与顺序约束

Service 可以指定顺序约束，代表某个 Service 代码块一定会先于/后于另一个代码块执行。

Vul 通过潜在事务调用构建代码块之间的顺序依赖图，并逐实例归纳成对子实例的调用顺序。



顺序依赖图的环代表：存在潜在组合逻辑环或不可解析的周期内依赖

这是保证高层行为描述具备硬件时序语义的核心机制

5. Vul 内置整数类型: $\text{Int}\langle N \rangle$

$\text{Int}\langle N \rangle$ 让用户在 C++ 行为描述中显式表达硬件整数的 位宽、截断、扩展、符号解释、位段访问和组合逻辑结构, 从而让仿真生成与 RTL 生成共享同一套语义基础。

1. **定宽度构造**: 明确的截断、扩展语义
2. **sint()**: 符号位视图, 改变解释方式但不改变底层bit
3. **$a + b \rightarrow \max(W_a, W_b) + 1$** : 显式保留进位避免溢出
4. **$a * b \rightarrow W_a + W_b$** : 完整乘法结果宽度
5. **不支持除法和取余**: 与硬件逻辑对齐
6. **at<hi, lo>, pick<wid>(base)**: bit slicing 与范围赋值
7. **Cat() / Repeat()**: 按位拼接与重复
8. **位运算, 按位Reduce**
9. **数值比较**

```
Int<8>  a = 0xFF;
Int<16> b = a;           // zext
Int<16> c = a.sint();    // xext

Int<9>  s = a + Int<8>(1);
Int<16> p = a * Int<8>(2);
Int<8>  low  = p.at<7, 0>();
p.at<15, 8>() = Int<8>(-1);
Int<27> = Cat(s, p);
```

6. Vul 内置寄存器组件: Register

Register 是 Vul 模块内部使用寄存器语义的标准接口，封装了模块本地的时序结构。

关键属性:

1. **成员类型**: 可以是基础类型或自定义的 struct 类型
2. **写端口数量**: 每周期能接受的写入数据个数，不同的写端口代表不同的仲裁优先级:

`count.setnext<1>(count + 1);` // 优先级1的端口，延迟写入自增1

`count.setnext<0>(0);` // 优先级0的端口，以最高优先级清零

3. **数组维度**: 寄存器数组在仿真和实现中包含独有的优化策略和不同的 API 定义:

读取 API: 无数组: `count` (隐式类型转换) 或 `count.get()`

数组: `count[idx]`

赋值 API: 无数组: `count.setnext<P>(v)`

数组: `count.setnext<P>(idx, v)`

7. Vul 内置存储组件库: Queue, BRAM, ROM

具有明确行为语义的存储组件应当是 “一等公民”

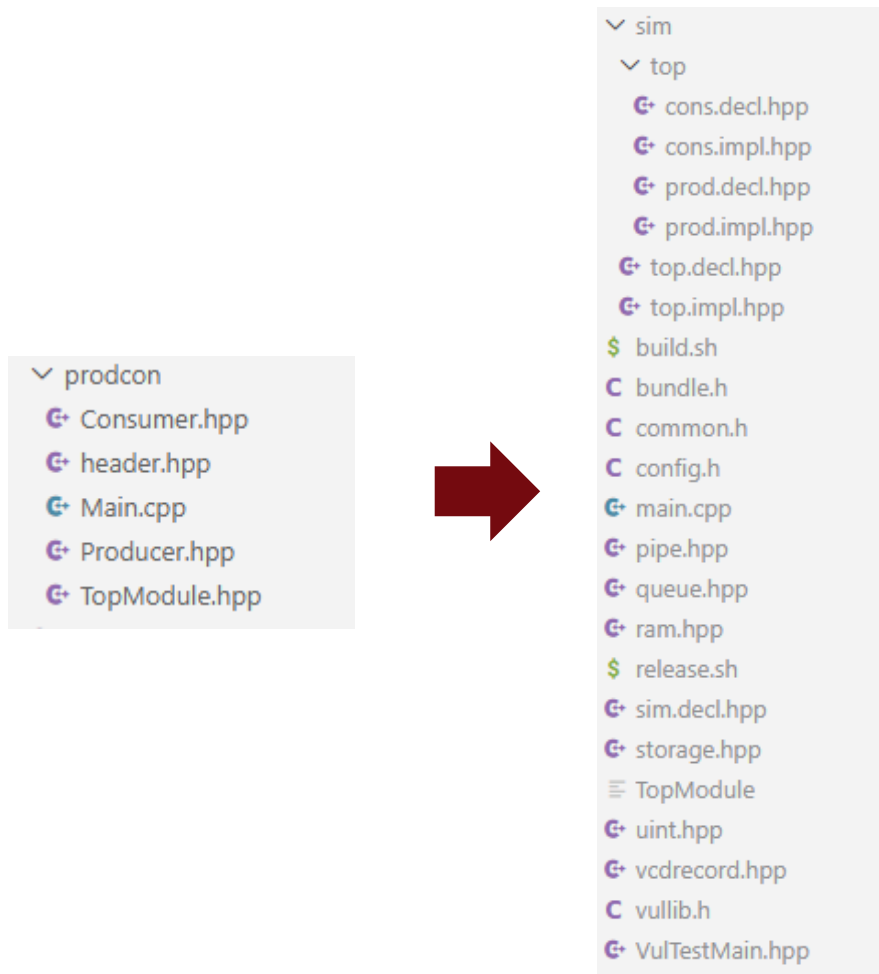
- **Queue:** 周期提交的硬件 FIFO 语义:
 - Queue<Type, Depth, EnqWidth, DeqWidth>
- **BRAM:** 显式端口定义的硬件 RAM 语义:
 1. BRAM<Type, AddrWidth, ReadPorts, WritePorts>: 多读写端口
 2. BRAM1RW<Type, AddrWidth>: 单端口读写共用
- **ROM:** 显式端口定义的硬件 ROM 语义:
 - ROM<DataWidth, AddrWidth, ReadPorts, InitFile>

所有存储组件均使用 “同步提交” 语义, 即所有状态修改均应用在下一周期开始前:

1. Queue.enqnext 和 deqnext 在下周期提交
2. BRAM/ROM.readreq 设置读端口地址, 读数据在下周期通过 readresp 获取
3. BRAM.write 设置写端口地址与数据, 写入在下周期生效

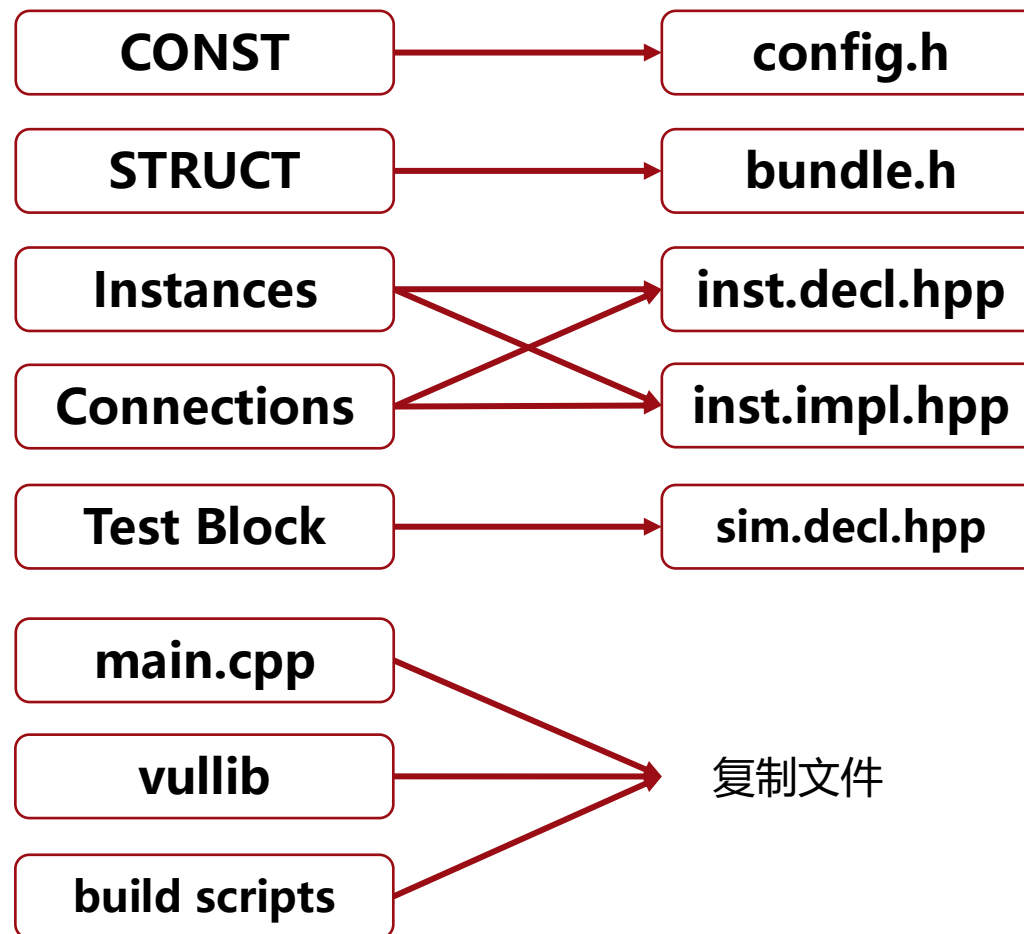
9. Vul后端：模拟器代码生成

以前文的 Prod-Cons 项目为例：



项目目录

可构建的C++代码



可使用标准 C++ 工具链编译运行

10. Vul后端：RTL生成

```
void Counter_tick() {  
    if (!output_can_push()) return;  
    uint32 val = count_get();  
    output_push(val);  
    count_setnext(val + 1, 1);  
}
```

```
void Counter_serv_clear(uint32 value) {  
    count_setnext(value, 0);  
}
```

用户编辑代码



结构化信息

代码合并/展开



```
bool output_valid;      // output port  
uint32 output_data;     // output port  
bool output_ready;      // input port  
  
bool serv_clear_valid;  // input port  
uint32 serv_clear_value; // input port  
  
uint32 count_value;     // internal register  
  
void _gen_tick() {  
    // default assignments  
    output_valid = false;  
    bool _gen_count_set0_valid = false;  
    uint32 _gen_count_set0_data;  
    bool _gen_count_set1_valid = false;  
    uint32 _gen_count_set1_data;  
  
    if (output_ready) { // if (output_can_push())  
        uint32 val = count_value; // uint32 val = count_get();  
        output_valid = true; // output_push(val);  
        output_data = val;  
        _gen_count_set1_valid = true; // count_set(val + 1, 1);  
        _gen_count_set1_data = count_value + 1;  
    }  
  
    if (serv_clear_valid) { // void Counter_serv_clear(uint32 value) {  
        _gen_count_set0_valid = true; // count_set(value, 0);  
        _gen_count_set0_data = serv_clear_value;  
    }  
  
    // arbitrate set requests  
    if (_gen_count_set0_valid && _gen_count_set1_valid) {  
        // both valid, priority to set0  
        count_value = _gen_count_set0_data;  
    } else if (_gen_count_set0_valid) {  
        count_value = _gen_count_set0_data;  
    } else if (_gen_count_set1_valid) {  
        count_value = _gen_count_set1_data;  
    }  
}
```

可HLS模块代码

RTLZZ



核心组合逻辑
代码

代码组合
模块连接



可综合RTL项目

代码组合
模块连接



RTL模板
生成

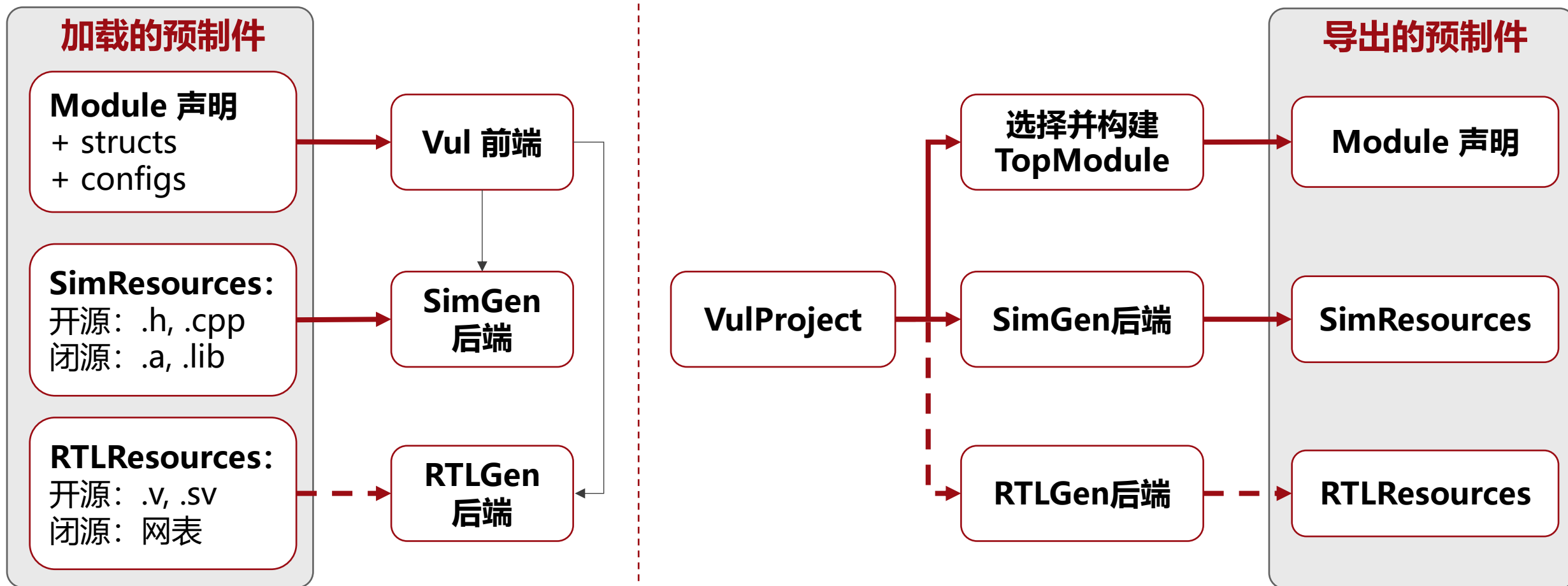


RTL模块框架

11. Vul 模块导入导出 (Working in progress)

预制件是可复用的 Vul 模型，包含一个只读的 Module 和依赖的 Configs/Structs。

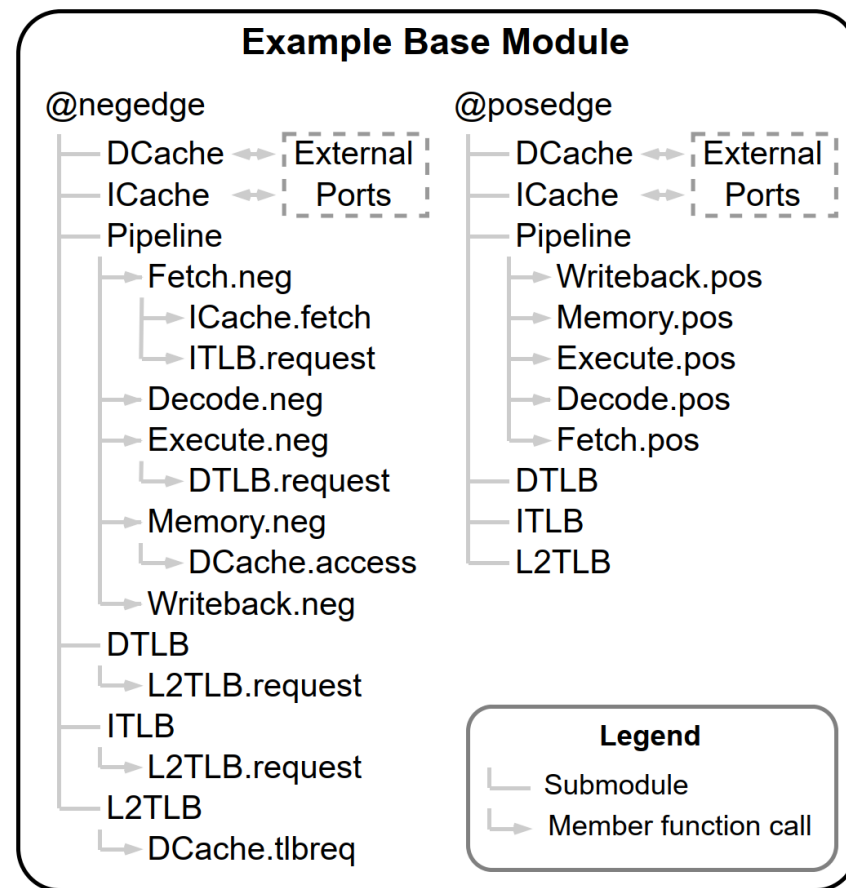
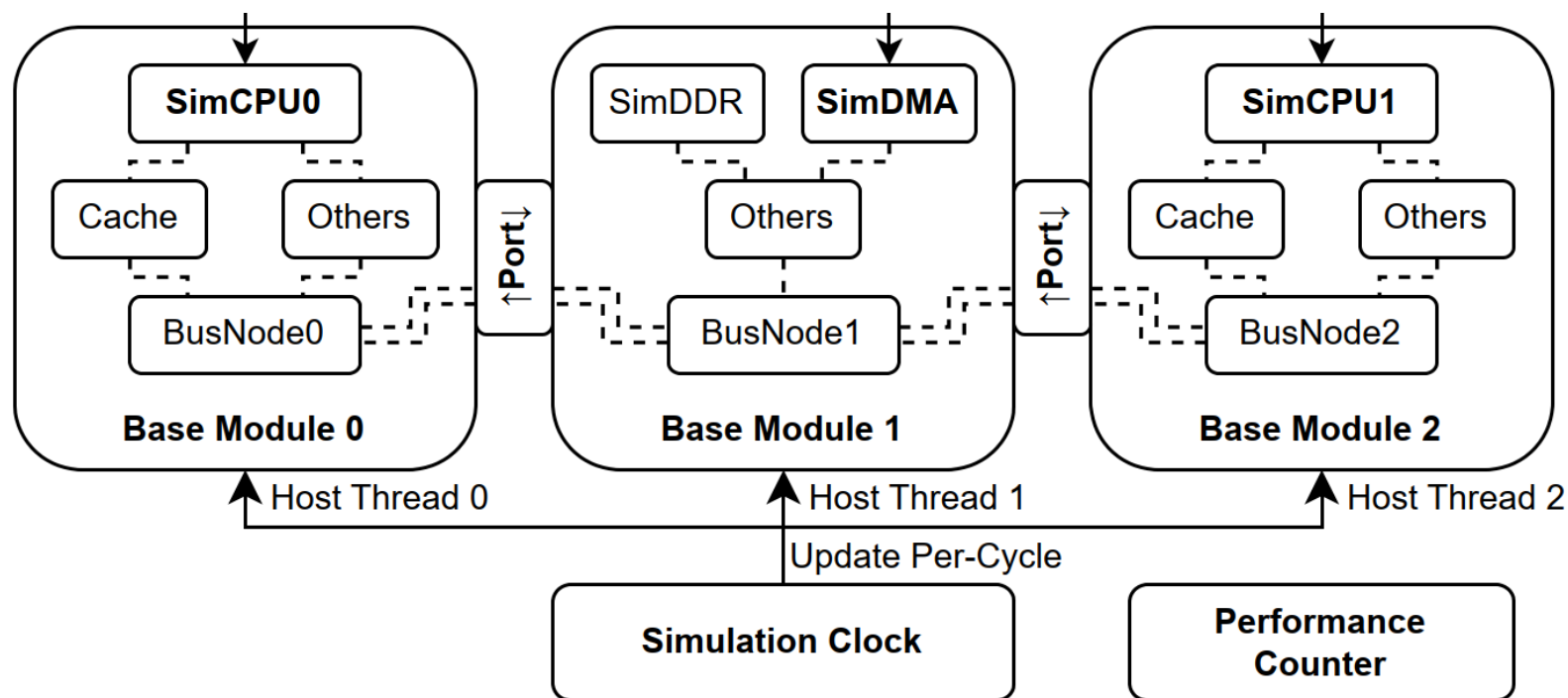
预制件不可直接编辑，但可以被 Vul 后端链接到生成的模拟代码或RTL中。



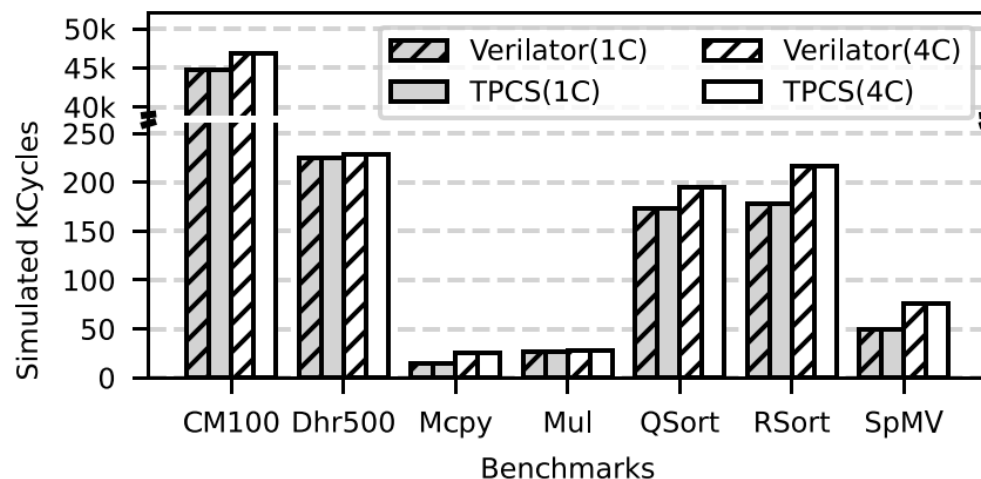
项目内部模块（合并多个实例和连接的蓝图）可以看作保存在项目内的，有源代码可编辑的预制件

12. 并行仿真方法

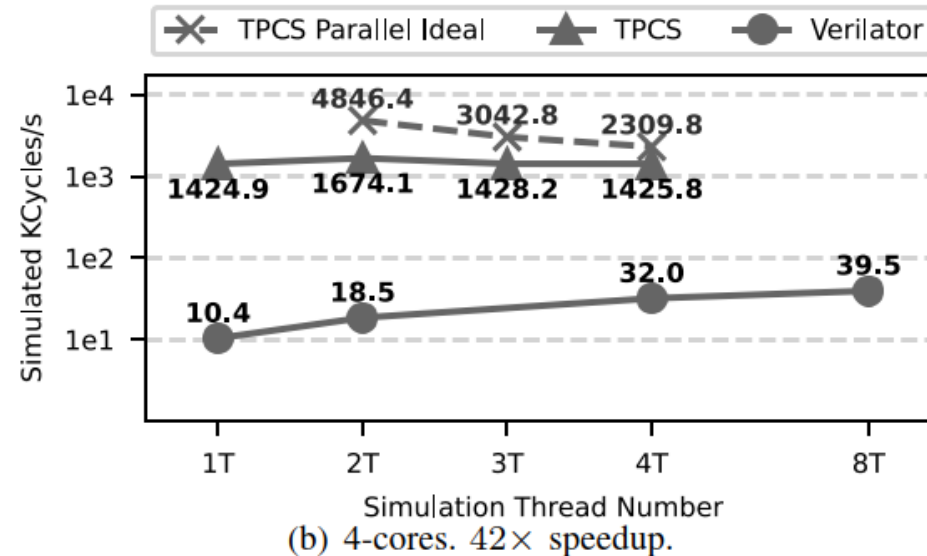
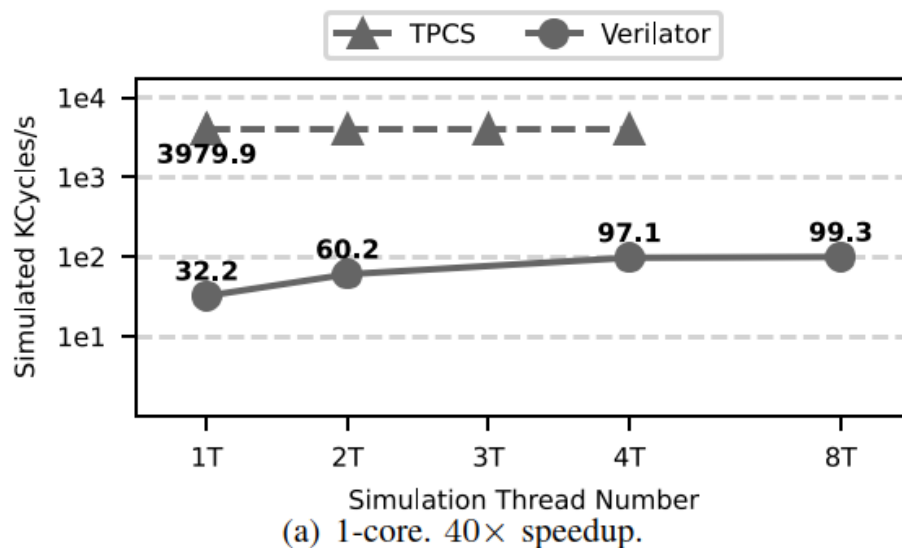
- 不同的硬件 Instance 可以在不同的模拟器线程中运行
- 每个模拟器线程按照 “Execute – Sync – Commit – Sync” 的工作流程不断循环
- Queue 语义可以跨线程
- 保证不同 enq 端和 deq 端之间不存在数据冲突



附. 与 Verilator 的性能比较测试



在1核/4核 RISC-V 顺序CPU SMP系统上达到了超过99%的周期级性能精度



相比于充分并行化的 Verilator, 我们的方案有超过40倍的模拟加速



山东大学
SHANDONG UNIVERSITY

4. 微架构无关的通用多核系统调用仿真方法

1. 简化迭代过程中的性能验证工作流程

系统调用模拟 Syscall Emulation，是一个常用于微架构模拟器的理念：

仅模拟部分处理器和缓存系统，直接加载用户态可执行文件，将系统调用代理到主机处理。

适用于早期不完整处理器设计下的性能评估。

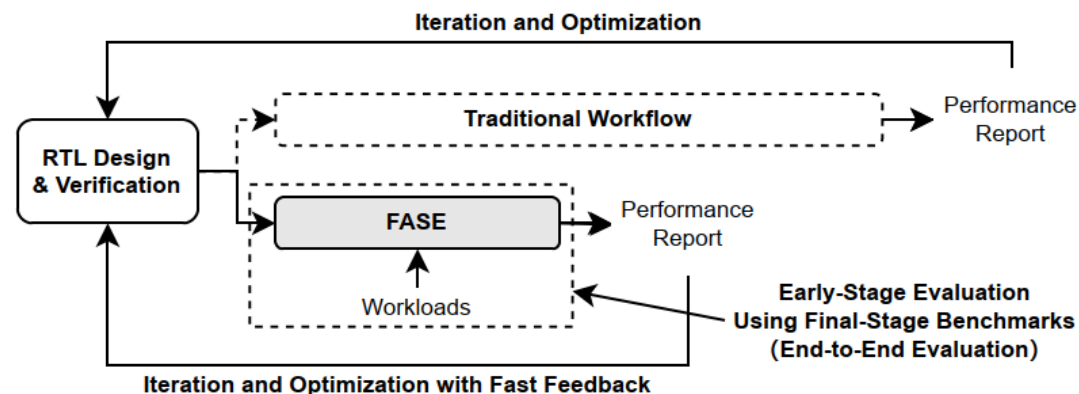
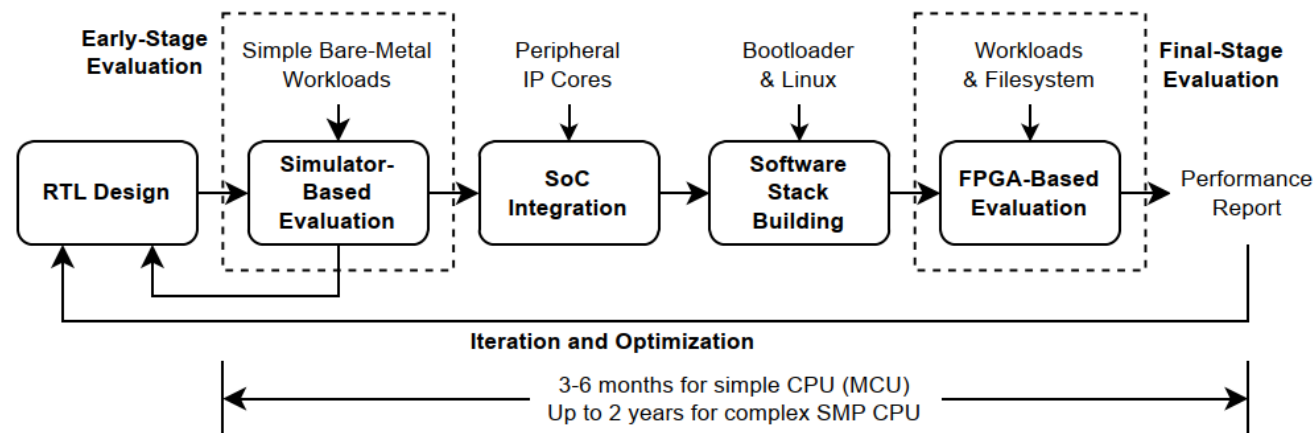
我们首次将该理念应用到了RTL模拟和FPGA上。

现有的早期性能验证方案Berkeley Proxy Kernel：

仅支持单核处理器，仅用于RTL模拟器。

我们的方案FPGA-Assisted Syscall Emulation：

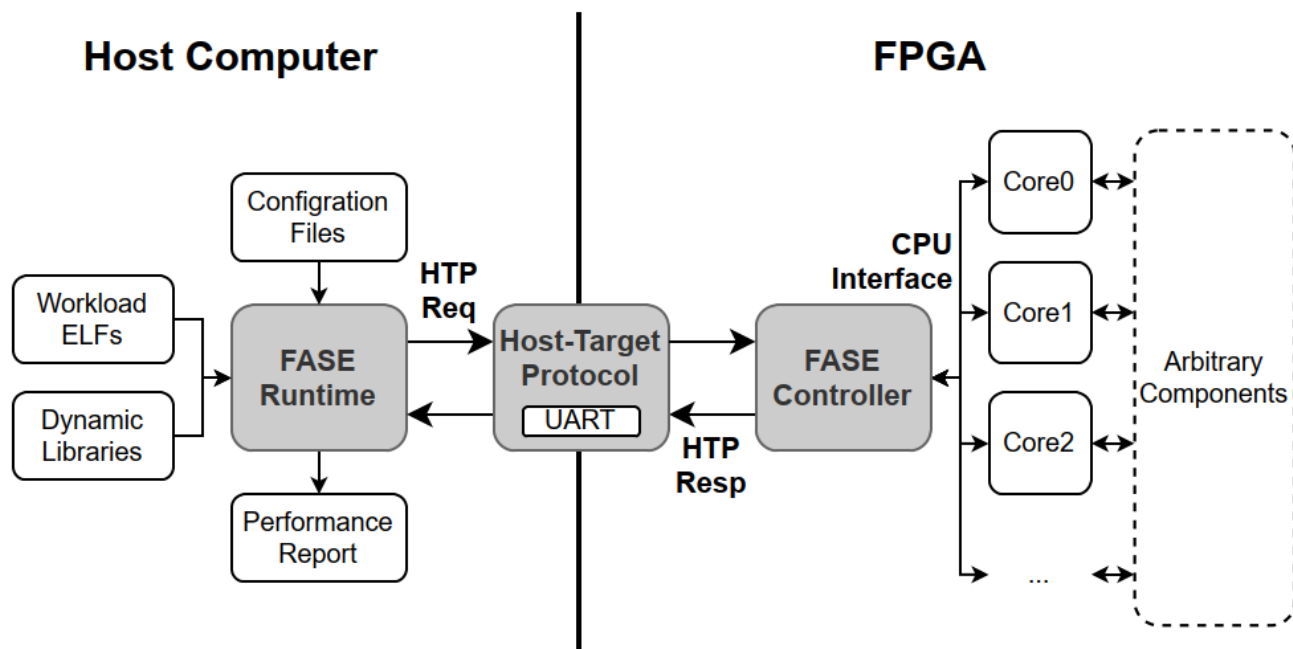
支持FPGA加速，支持多核处理器



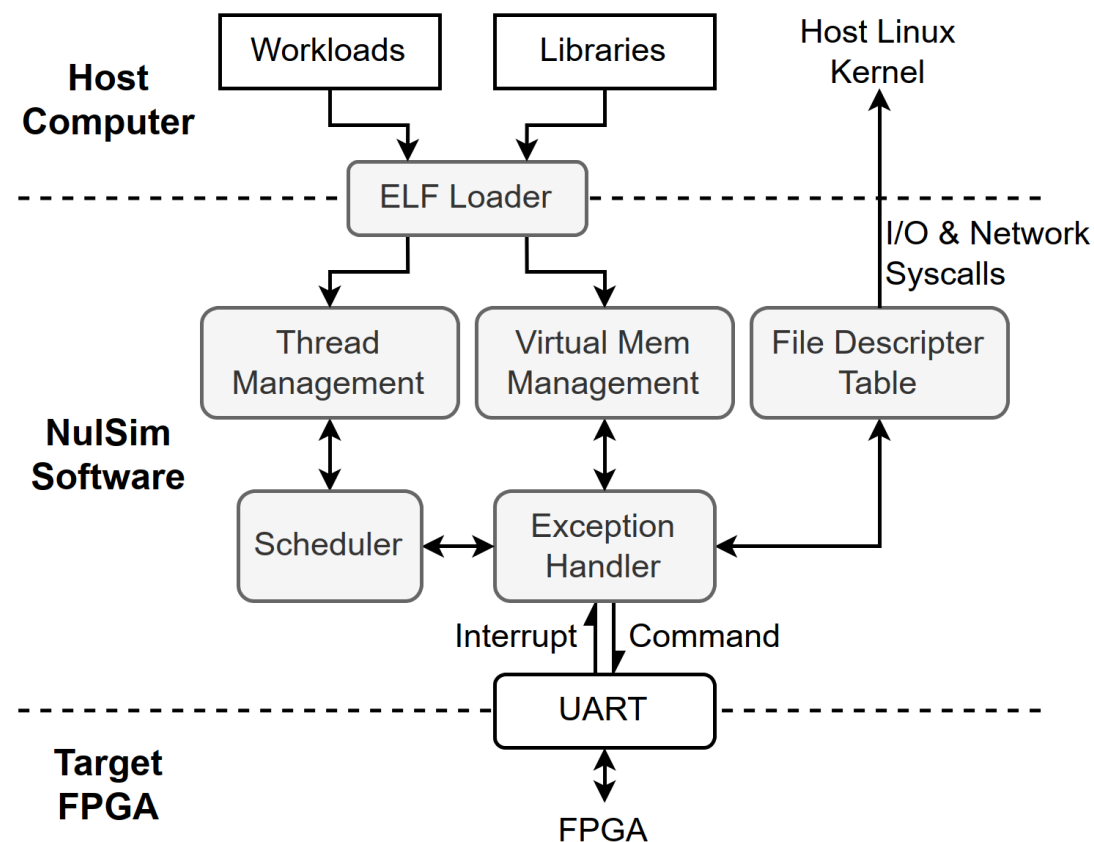
SE模拟简化了性能验证的工作流程，加快了性能反馈

2. FASE (FPGA-Assisted Syscall Emulation)

FPGA上的控制器通过串口与主机通讯，通过CPU接口控制每一个处理器核心。
主机Runtime软件从UART等待处理器中断事件并处理。

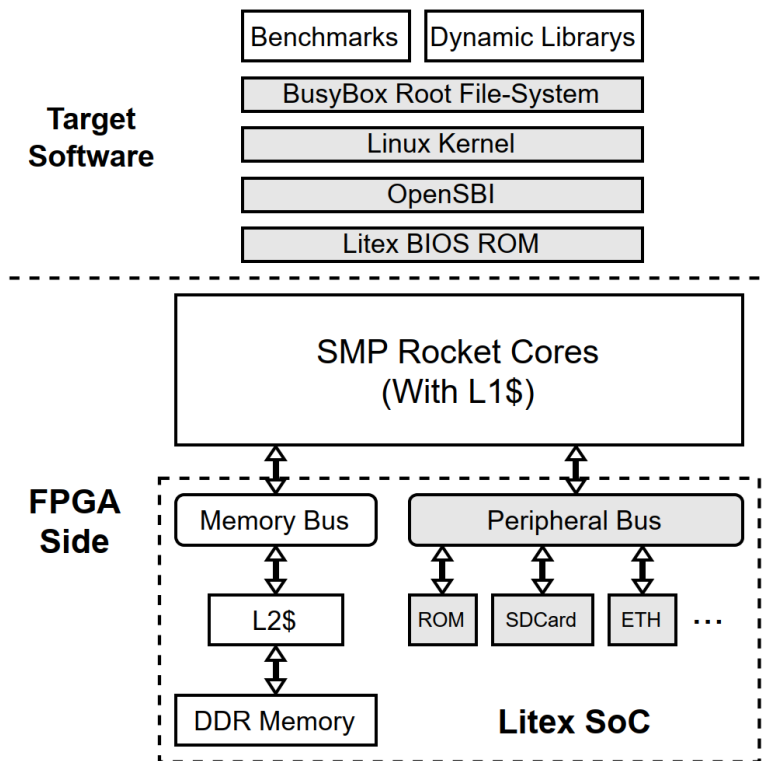
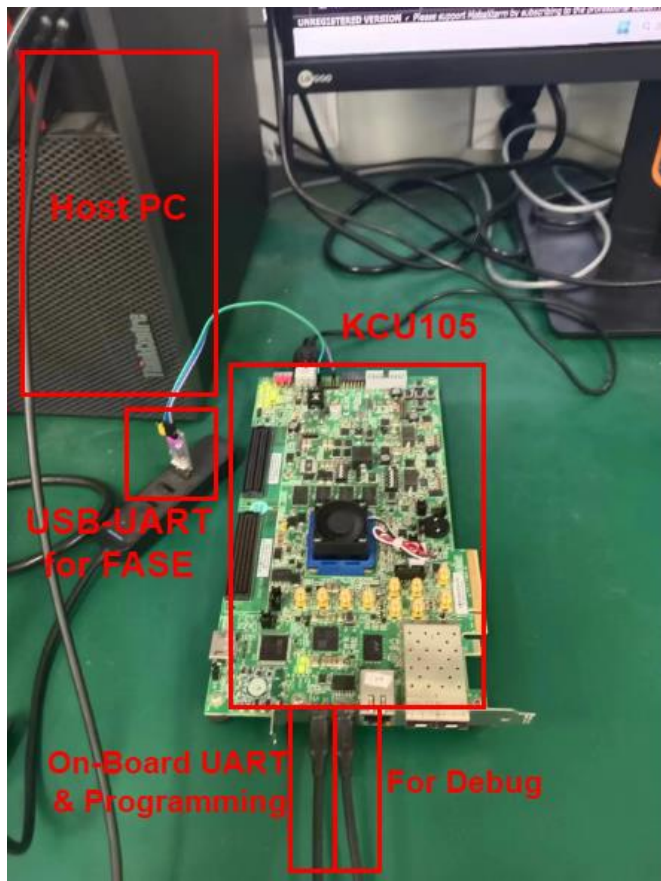


FPGA上的系统调用模拟总览架构

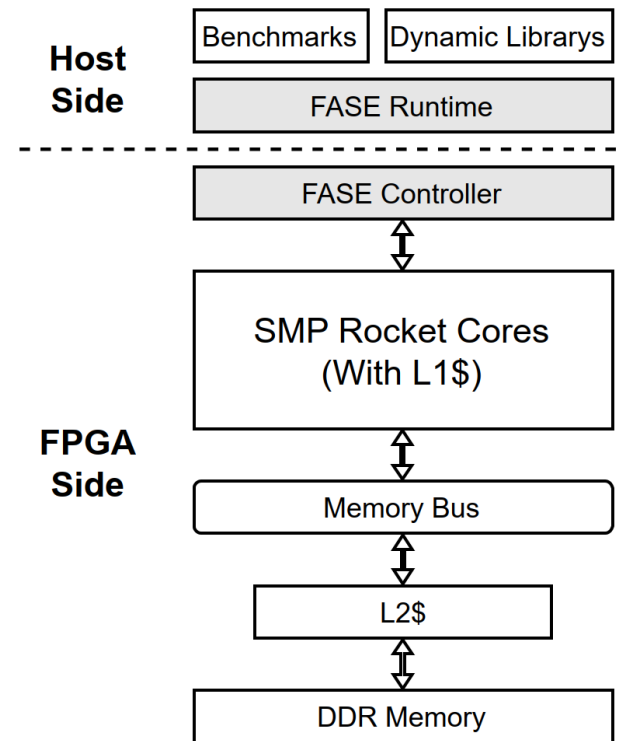


主机软件架构

4. FASE的实验验证



(a) Baseline full-system SoC

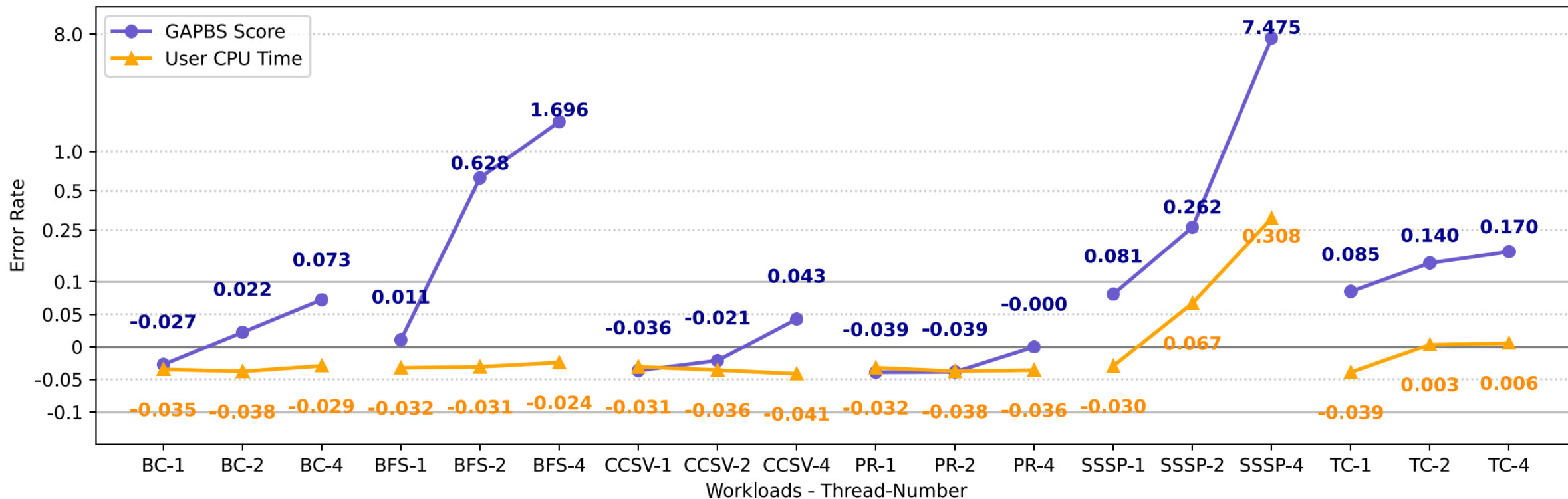


(b) FASE without SoC

在KCU105 FPGA上，使用开源RV64处理器Rocket 4核SMP 进行验证

对比组使用基于LiteX构建的 SoC+Linux Full-System 和 ProxyKernel+Verilator 运行基于OpenMP的开源Benchmarks GAPBS, 和CoreMark (用于对比PK)

4. FASE的实验验证

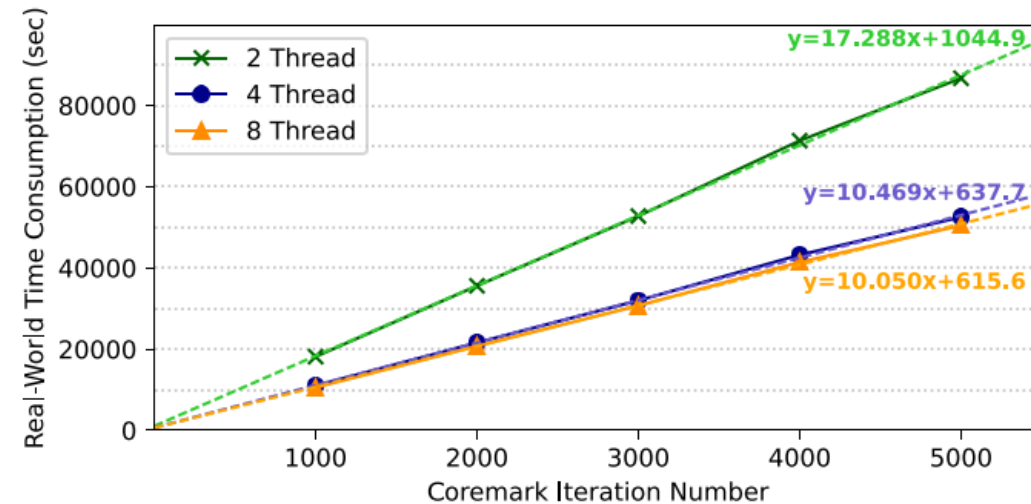
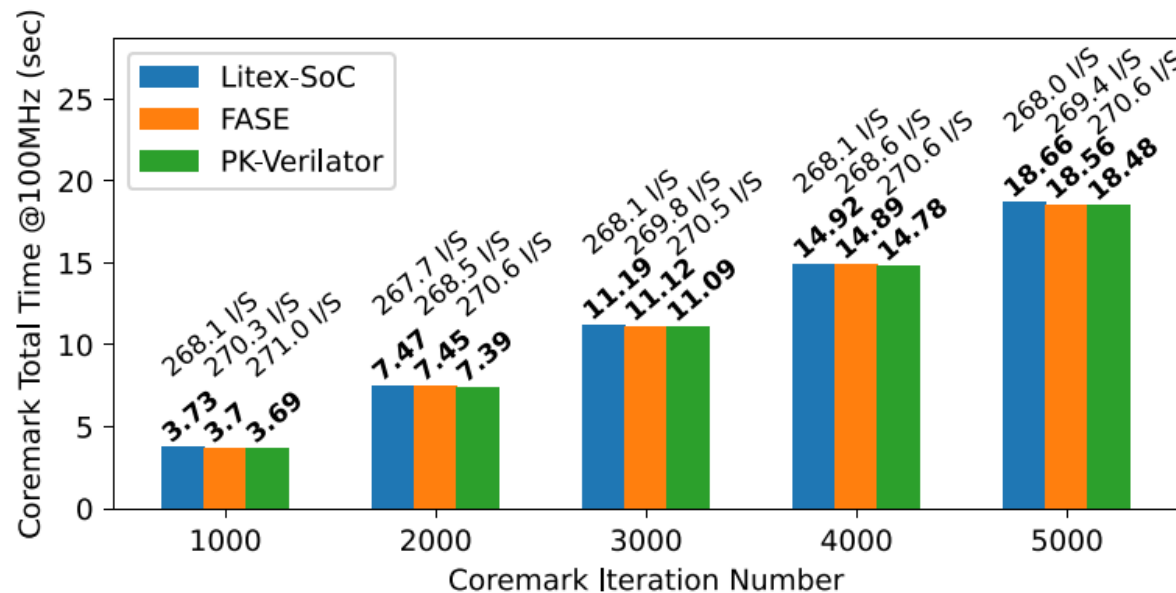


在6个基准测试项的1/2/4线程测试中，FASE与基线LiteX得到的性能结果误差率。

GAPBS Score: 基准测试得分。 User CPU Time: CPU消耗在用户态的总时间。

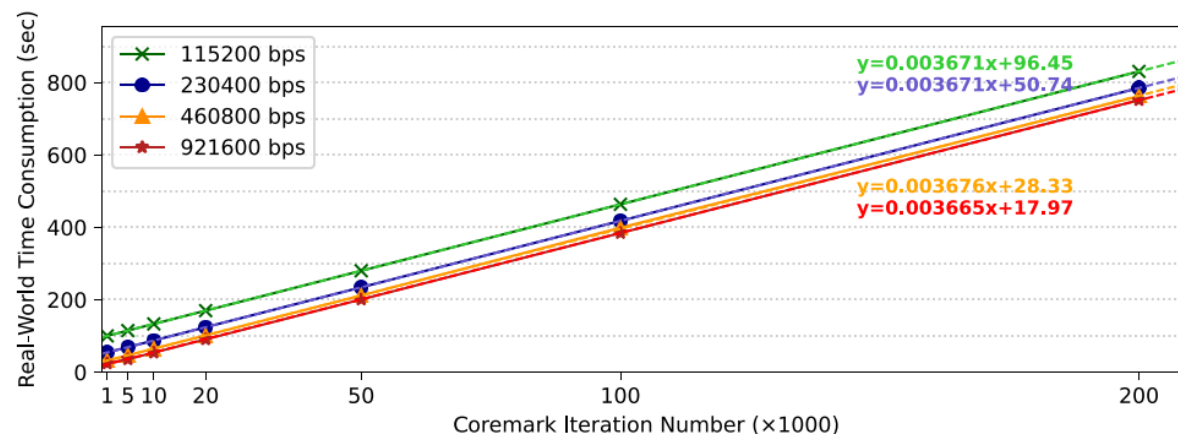
单线程平均误差率小于5%，多线程平均误差率小于10%

4. FASE的实验验证



(a) PK baseline, where “2 Thread” indicates execution with 2 Verilator simulation threads.

- 在单核CoreMark中，性能准确率>99%。
- 与 Verilator+PK 相比，效率提高>x2000，由于应用FPGA加速



(b) FASE, where “115200 bps” indicates a UART baud rate of 115200.

5. 工作状态

- 开源代码: <https://github.com/meng-cz/fase-rv64>
- 目前工作有:
 - 重构硬件控制器代码, 降低控制器本身的 FPGA 资源消耗
 - 测试适配香山 CPU 核心
 - 给出简单的硬件 example 用于快速部署验证
 - 测试使用 PCIE-XDMA 作为 Host-Target-Protocol 的物理信道

谢谢!

